웹 개발자 부트캠프 과정

CodingOn

Spring 과 Database

스프링 데이터베이스 접근

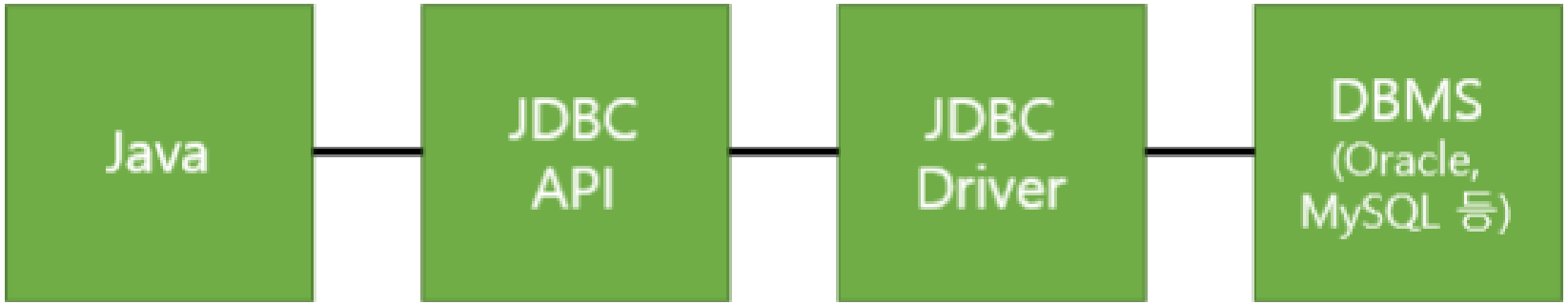
- 스프링에서 데이터베이스에 접근할 수 있는 방법에는 여러가지가 있음

- **다른 라이브러리와 연동**
  - 스프링 외부의 데이터베이스 접근 라이브러리를 스프링과 통합하여 사용하는 방식
  - ex. MyBatis, Hibernate
- **스프링 데이터 (Spring Data)**
  - 스프링에서 제공하는 프로젝트
  - ex. Spring Data JPA, Spring Data JDBC, Spring Data MongoDB
- **스프링 JDBC**
  - 스프링 프레임워크에서 제공하는 기능

JDBC

JDBC 란?

- Java DataBase Connectivity
- 자바 언어와 DB를 연결해주는 통로로, 자바에서 데이터베이스에 접근할 수 있도록 해주는 자바 API



- 단점: 반복적이고 지루한 코드 작성이 필요하며, SQL 예외 처리, 리소스 관리(연결, 문, 결과 집합 닫기 등) 등의 복잡한 작업을 직접 처리

Spring JDBC

- 정의
  - JDBC의 복잡성을 줄이기 위한 Spring 프레임워크의 일부
  - 데이터베이스 작업을 단순화하고 코드의 양을 줄여줌

- 작동 방식
  - 기존 JDBC 코드의 많은 부분을 추상화하고, 템플릿 디자인 패턴을 이용

SQL Mapper 와 ORM

SQL Mapper와 ORM

- Spring은 대표적으로 두 가지 유형의 데이터베이스와의 상호작용 방식이 존재
  1. SQL Mapper
  2. ORM (Object-Relational Mapping)

- 공통점: 두 방식 모두 Java 객체와 데이터베이스 간의 상호 작용

- 각 방식마다 장단점이 있으므로 프로젝트의 특성과 요구사항에 따라 적절히 선택하여 사용함

SQL Mapper와 ORM 차이점 (1)

- **SQL Mapper**
  - 목적: SQL 쿼리와 Java 메서드/객체 간의 매핑을 중심으로 함
  - 개발자는 SQL 쿼리를 직접 작성하고, 해당 쿼리의 결과를 Java 객체에 매핑함
  - 특징
    - 동적 SQL 생성, 조건문, 반복문 등의 SQL 작성에 있어서 유연
    - SQL 쿼리의 세밀한 튜닝이 필요할 때 주로 사용
    - 데이터베이스 중심의 개발 방식

장점	단점
복잡한 쿼리 작성 및 성능 최적화 용이	객체와 테이블간 불일치 문제 해결에 추가 작업 필요
SQL이 익숙한 개발자들이 쉽게 적응 가능	DBMS 변경 시 SQL 수정 필요한 경우 있음

  - ex. MyBatis, JDBCTemplate

SQL Mapper와 ORM 차이점 (2)

- **ORM (Object-Relational Mapping)**
  - 목적: Java 객체와 데이터베이스 테이블 간의 관계 매핑
  - Java 객체(Entity)가 데이터베이스의 테이블과 어떻게 매핑 될지를 정의함
  - 특징
    - 객체 중심의 개발 방식
    - 대부분의 CRUD 연산에 대한 SQL을 자동으로 생성
    - 개발자는 객체 지향적으로 데이터를 다룰 수 있음

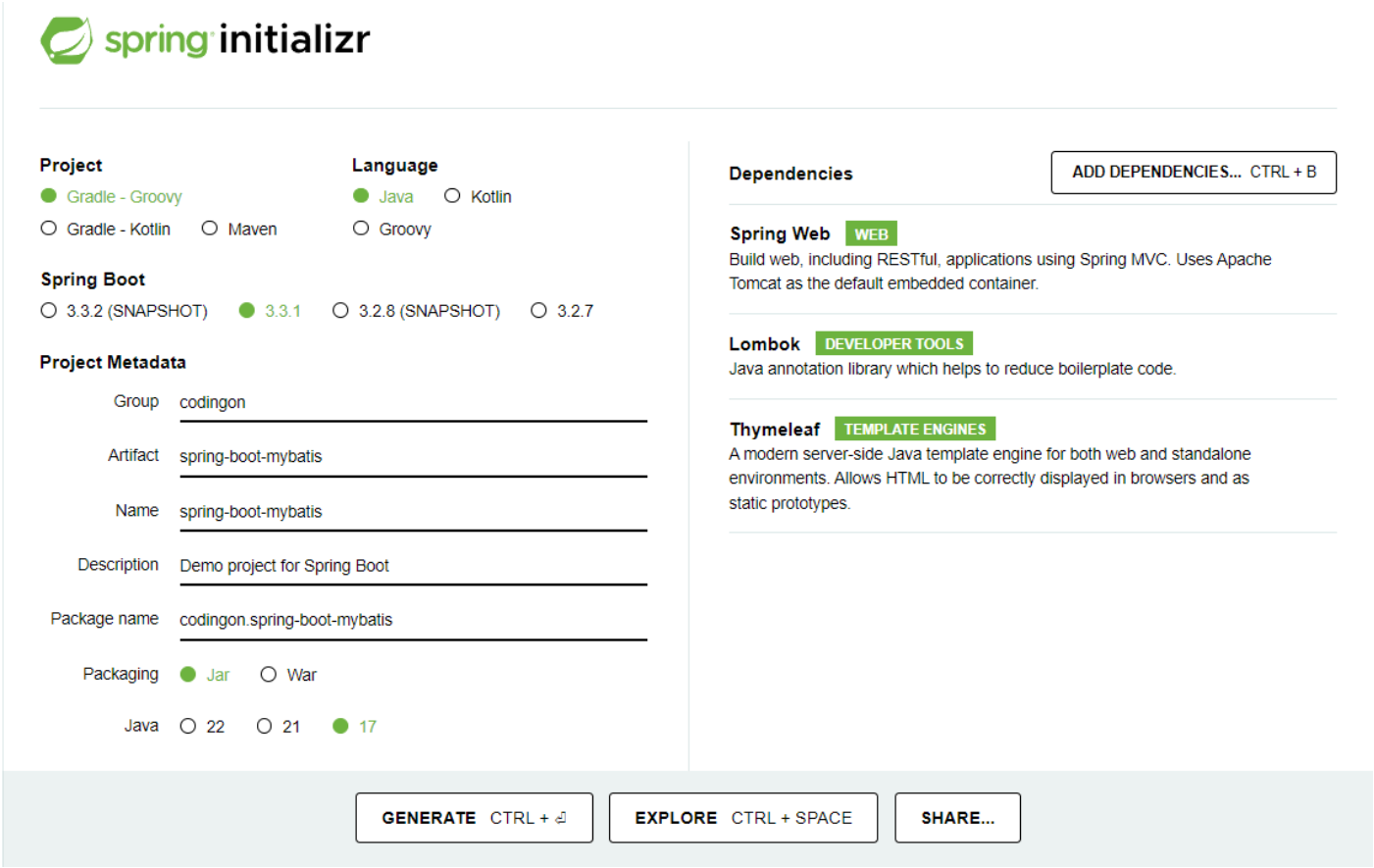
장점	단점
생산성 향상 (반복적인 CRUD 작업 감소)	높은 학습 곡선
유지보수 용이성 증가	복잡한 쿼리는 성능 이슈 가능성
DBMS에 대한 종속성 감소	자동 생성된 SQL의 최적화의 어려움

  - 예: Hibernate, JPA

MyBatis

Spring Boot 프로젝트 생성하기

https://start.spring.io/



잠깐! build.gradle 수정

- MyBatis 3.0은 Spring Boot 3.0 ~ 3.1 버전 까지만 지원 (2024년 7월 기준)
  - 즉, MyBatis 3.0이 Spring Boot 최신 버전과 호환되지 않음
  - 실제 프로젝트에서 기술 스택을 선택할 때 고려해야 하는 중요한 요소로 최신 기술을 사용하고 싶다면 호환성을 신중히 검토해야 함

The MyBatis-Spring-Boot-Starter requires following versions:

MyBatis-Spring-Boot-Starter	MyBatis-Spring	Spring Boot	Java
3.0	3.0	3.0 - 3.1	17 or higher
2.3	2.1	2.5 - 2.7	8 or higher

https://mybatis.org/spring-boot-starter/mybatis-spring-boot-autoconfigure/

build.gradle 수정

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.1.5'
    id 'io.spring.dependency-management' version '1.1.4'
}
```

MyBatis 란?

- SQL Mapper 로써 JDBC로 처리하는 상당 부분의 코드와 파라미터 설정 및 결과 매핑을 대신해준다.

- 객체 지향 언어인 자바와 관계형 데이터베이스 프로그래밍을 좀 더 쉽게 할 수 있도록 도와주는 프레임워크

MyBatis 특징

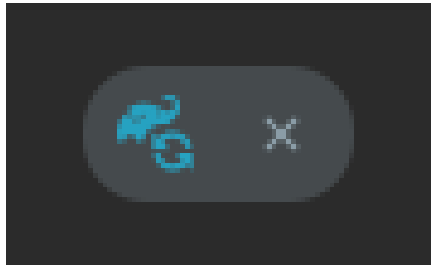
- 쉬운 접근성과 코드의 간결함
  - JDBC의 모든 기능을 MyBatis 에서 사용 가능
  - 복잡한 JDBC 코드를 걷어내 깔끔한 소스코드 유지 가능
- SQL 문과 프로그래밍 코드의 분리
  - SQL 문과 프로그래밍 코드가 분리되어 있어 SQL에 변경이 있을 때 다시 컴파일할 필요가 없다.
- 다양한 프로그래밍 언어로 구현 가능
  - Java, C#, .NET, Ruby

MyBatis 시작하기

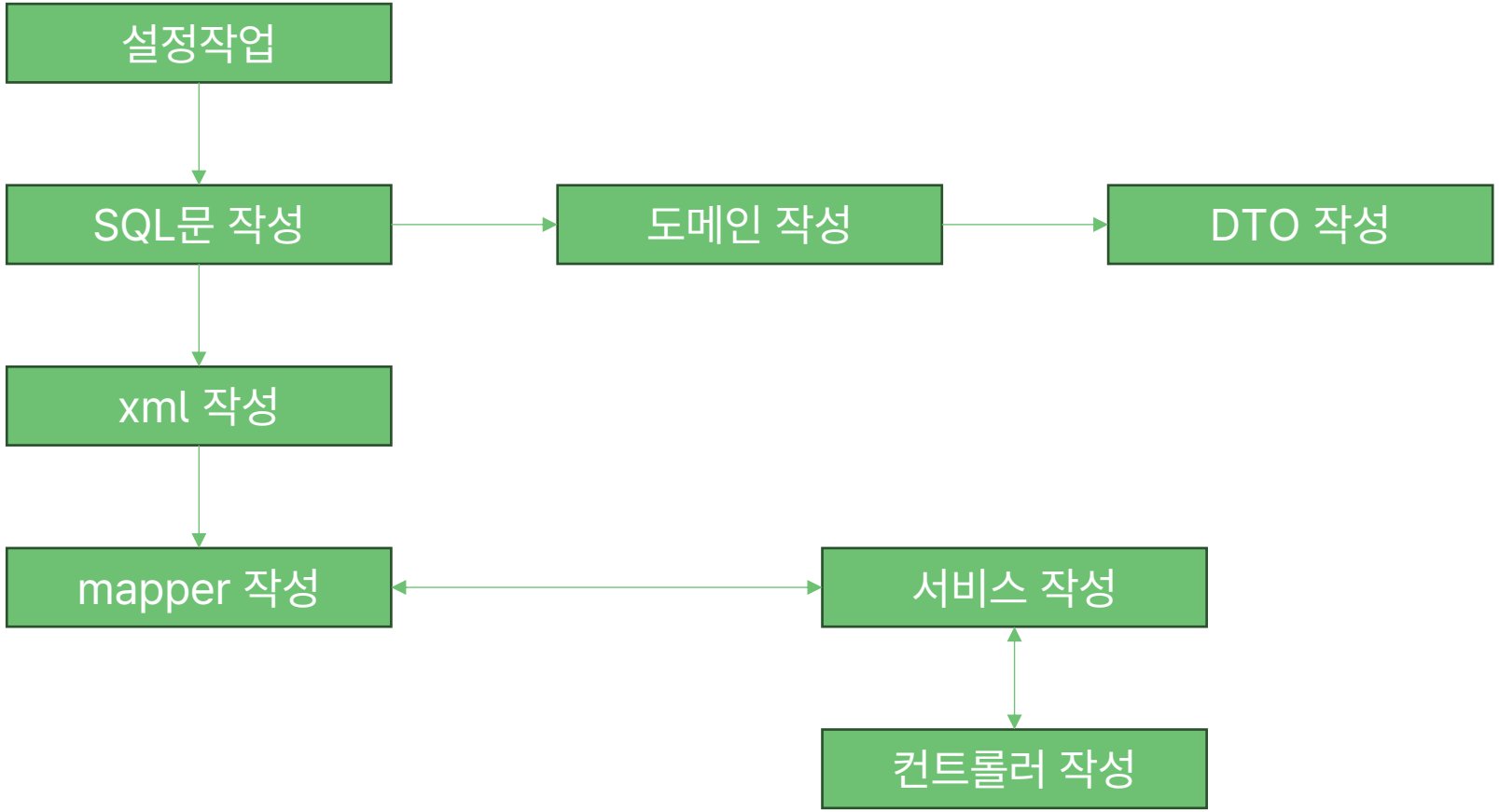
- build.gradle 파일의 dependencies 항목 추가

```
implementation 'org.mybatis.spring.boot:mybatis-spring-boot-starter:3.0.2'
runtimeOnly 'com.mysql:mysql-connector-j'
```

- MyBatis를 사용하기 위해 “Load Gradle Changes” 버튼 클릭
  - Gradle 프로젝트 설정이 변경되었을 때 이를 IDE에 반영!



흐름을 파악하기!



application.properties 수정

- (복습) Spring Boot 애플리케이션의 설정을 관리하는 중요한 파일
- 런타임 시 다양한 환경에서 동작할 수 있도록 필요한 옵션들을 제공하자!

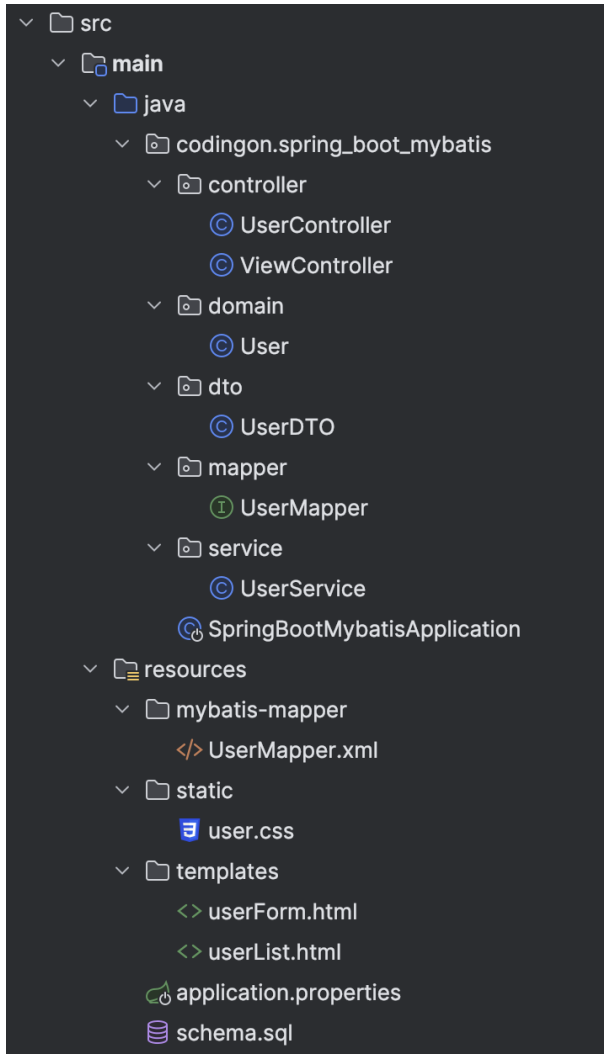
```
# 1. 데이터베이스 설정
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/[DataBase이름]?useUnicode=yes&characterEnc
oding=UTF-8&allowMultiQueries=true&serverTimezone=Asia/Seoul
spring.datasource.username=root
spring.datasource.password=1234


# 2. MyBatis 설정
mybatis.type-aliases-package=[베이스패키지명].mapper
mybatis.mapper-locations=[폴더명]/*.xml
```

* 데이터베이스명은 개발 환경마다 달라짐!

설정 끝! 개발 시작

- 패키지 및 파일 생성
  - controller.ViewController 클래스
  - controller.UserController 클래스
  - domain.User 클래스
  - dto.UserDTO 클래스
  - mapper.UserMapper 인터페이스
  - service.UserService 클래스

  - resources/mybatis-mapper/UserMapper.xml 파일
  - resources/schema.sql 파일



resources/schema.sql

```sql
create database codingon default character set utf8 default collate utf8_general_ci;
use codingon;

-- users 테이블 생성
create table users (
                id bigint auto_increment primary key,
                username varchar(50) not null,
                email varchar(100) not null,
                created_at timestamp default current_timestamp
);

-- 더미 데이터 삽입
insert into users (username, email) values
                        ('john_doe', 'john.doe@example.com'),
                        ('jane_smith', 'jane.smith@example.com'),
                        ('bob_johnson', 'bob.johnson@example.com');

select * from users;
```

codingon 데이터베이스 생성 후, user 테이블 생성

domain.User 클래스

```
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class User {
    private Long id;
    private String username;
    private String email;
    private String createdAt;
}
```

user 테이블 컬럼과 필드가 일치해야 함

dto.UserDTO 클래스

```
import lombok.*;

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class UserDTO {
    private Long id;
    private String username;
    private String email;
    private int no;
}
```

domain.User와 다르게 no 필드 추가
실제 테이블에 존재하는 컬럼은 아니지만, 서비스 로직에서 no 정보를 활용할 예정

mapper.UserMapper 인터페이스

```
import codingon.spring_boot_mybatis.domain.User;
import org.apache.ibatis.annotations.Insert;
import org.apache.ibatis.annotations.Mapper;

import java.util.List;

@Mapper
public interface UserMapper {

    // case1. 어노테이션 기반 매퍼
    //@Select("SELECT * FROM users")
    //List<User> findAll();


    // case2. XML 기반 매퍼
    List<User> findAll();

}
```

resources/…/UserMapper.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE mapper PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN" "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="codingon.spring_boot_mybatis.mapper.UserMapper">
    <!-- XML 기반의 MyBatis 매퍼 사용 -->
    <select id="findAll" resultType="codingon.spring_boot_mybatis.domain.User">
        SELECT * FROM users
    </select>
</mapper>
```

UserService 클래스

```java
@Service
public class UserService {
    @Autowired
    private UserMapper userMapper;

    public List<UserDTO> getAllUsers() {
        List<User> users = userMapper.findAll();
        List<UserDTO> userDTOs = new ArrayList<>();
        for (User user : users) {
            UserDTO userDTO = convertToDto(user);
            userDTOs.add(userDTO);
        }
        return userDTOs;
    }

    private UserDTO convertToDto(User user) {
        UserDTO dto = new UserDTO();
        dto.setId(user.getId());
        dto.setUsername(user.getUsername());
        dto.setEmail(user.getEmail());
        dto.setNo((int) (user.getId() + 100));
        return dto;
    }
}
```

UserController 클래스

```java
package codingon.spring_boot_mybatis.controller;

import codingon.spring_boot_mybatis.dto.UserDTO;
import codingon.spring_boot_mybatis.service.UserService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/users")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping
    public List<UserDTO> listUsers() {
        return userService.getAllUsers();
    }
}
```

ViewController 클래스

```java
package codingon.spring_boot_mybatis.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class ViewController2 {

    @GetMapping("/")
    public String redirectToUsers() {
        return "redirect:/users";
    }

    @GetMapping("/users")
    public String listUsers() {
        return "userList";
    }

}
```

API 정리

- 전체 유저 조회: GET /api/users
- 새 유저 생성: POST /api/users
- 특정 ID 유저 조회: GET /api/users/{id}
- 특정 ID 유저 수정: PUT /api/users/{id}
- 특정 ID 유저 삭제: DELETE /api/users/{id}

나머지 API 도 완성해보자!

MyBatis 동작 원리

- 애플리케이션 시작 시 SqlSessionFactoryBuilder 가 설정 파일을 참고해 SqlSessionFactory 생성
  - SqlSessionFactory : 데이터베이스와의 연결과 SQL의 실행에 대한 모든 것을 가진 중요한 객체
- DB 작업 시 SqlSessionFactory 가 SqlSession 생성
  - SqlSession : Connection을 생성하거나 원하는 SQL을 전달하고, 결과를 Return 해주는 객체
- 생성된 SqlSession을 참고해 mapper 인터페이스 호출
- Mapper 가 SqlSession을 호출해 SQL 실행 후 결과 Return

실습. 게시판 시스템

- C : 게시판 글 작성
- R : 게시판 글 조회 (조건 추가해서 작성자가 a일 때)
- U : 게시판 글 업데이트
- D : 게시판 글 삭제

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
title	varchar(20)	NO		NULL	
content	varchar(100)	NO		NULL	
writer	varchar(10)	NO		NULL	
registered	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

- 지난 시간에 배운 API와 오늘 배운 MyBatis 를 이용해 DB 와 연동되는 게시판 시스템 제작해보기!